



Program Sarjana, Departemen Teknik Komputer, FTEIC-ITS

Proyek “2EZ4U APP”

Studi Kasus Proyek Akhir
Struktur Data & Analisa Algoritma (EC234303)
Dosen: Ir. Arta Kusuma Hernanda, S.T., M.T.

Project yang akan dikembangkan adalah **engine backend layanan antar makanan/barang** seperti GoFood/GrabFood/ShopeeFood versi mini.

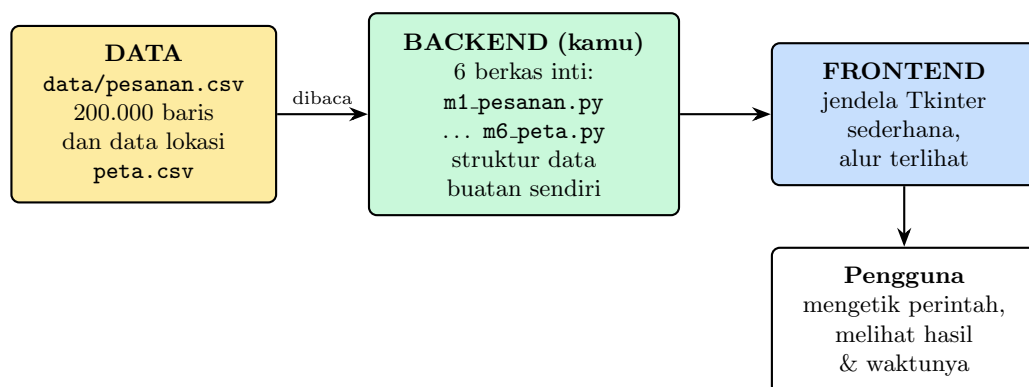
Bentuk	individu	Bahasa	Python 3.11+
Bobot	35% nilai akhir	Submission	repo GitHub
Mulai	minggu 1	Demo	Progres(M9) Final(M17)

1 Apa yang Kamu Develop?

Bayangkan kamu akan mengembangkan aplikasi delivery service dengan trafik user yang cukup tinggi. Aplikasi yang kamu kembangkan terdiri atas tiga layer:

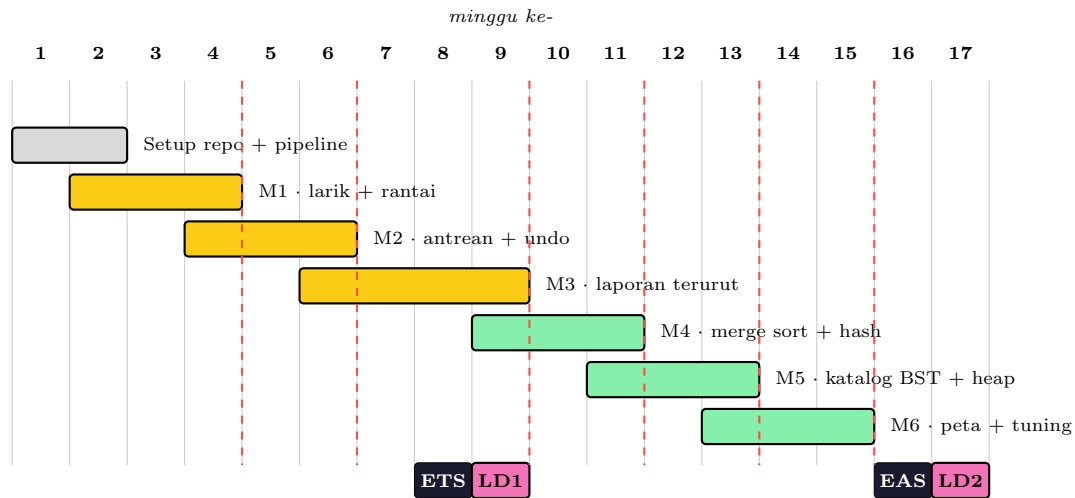
1. **Data** (data/): 200.000 data CSV yang diasumsikan sebagai data dalam sistem database aplikasi kalian.
2. **Backend** (backend/): **enam fitur utama**, satu fitur untuk tiap milestone, berisi struktur data dan algoritma buatanmu.
3. **Frontend** (frontend/): satu jendela **Tkinter** sederhana yang memanggil backend dan **memperlihatkan alurnya** kepada pengguna.

Tampilannya boleh sangat sederhana. Yang dinilai paling besar bukan terkait estiteka UI, melainkan proses implementasi **struktur data dan algoritma**.



2 Peta Perjalanan Satu Semester

2.1 Timeline (Gantt)



LD1 = sesi Live Demo Progress (mgg 9) · LD2 = sesi Live Demo Final (mgg 17)

2.2 Ringkasan milestone

#	Fitur yang terlihat di antarmuka	Berkas backend
M1	array vs linked list: lihat, tambah, hapus pesanan	m1_pesanan.py
M2	implementasi queue dan stack (untuk undo)	m2_antrean.py
M3	penerapan berbagai macam jenis sort	m3_laporan.py
M4	penerapan berbagai macam jenis search	m4_pencarian.py
M5	menu per rentang harga; dispatch prioritas	m5_katalog.py
M6	map dan navigasi rute	m6_peta.py

3 Aturan Main

1. **Project Individu.**
2. **Dilarang menggunakan instant function yang tersedia.** Seperti: dict, set, sorted, .sort(), heapq, bisect, collections.*, literal {}, dan comprehension dict/set. **Boleh:** list sebagai array primitif, tuple, aritmetika, I/O.
3. **Data disediakan dalam format CSV dengan jumlah 200.000 (data.csv) dan data peta.csv.**
4. **UI python desktop seperti Tkinter lib atau sejenisnya.**
5. **Upload project folder ke repository Github,** Wajib menyertakan README.md yang berisi deskripsi project, cara menjalankan, diagram, screenshot, video demo dan catatan penting lainnya. Link repo Github akan dikumpulkan melalui CEITS/ENIGMA.

4 Struktur Repositori

Gambar struktur repo github.

```
2ez4u/
|-- README.md          <- LAPORANMU
|
|-- data/
|   |-- peta.csv
|   |-- pesanan.csv
|
|-- backend/
|   |-- m1_pesanan.py    M1: Larik (array) + Rantai (linked list)
|   |-- m2_antrean.py    M2: AntreanMelingkar + Tumpukan (undo)
|   |-- m3_laporan.py    M3: insertion sort + binary search
|   |-- m4_pencarian.py  M4: PetaHash + merge sort
|   |-- m5_katalog.py    M5: Katalog (BST) + MinHeap (prioritas)
|   |-- m6_peta.py       M6: Peta (graf) + BFS + Dijkstra
|
|-- frontend/          <- ANTARMUKA
|   |-- ui.py
|
|-- app.py              "python app.py" main application
```

4.1 Contoh OOP dalam backend

```
# ----- backend/m1_pesanan.py -----
class Array:
    def append(self, v): ...
    def insert(self, i, v): ...
    def get(self, i): ...
    ....

class LinkList:
    def append(self, v): ...
    def get(self, i): ...
    ....

# ----- backend/m2_antrean.py -----
class queue:
    def enqueue(self, x): ...
    def dequeue(self): ...
    ...

class stack:
    def push(self, x): ...
    def pop(self): ...
    ...

# ----- backend/m3_sort.py -----
def binary_search(larik, target): ...
....

# ----- backend/m4_pencarian.py -----
class PetaHash:
    def hash_teks(self, s): ...
    def put(self, k, v): ...
    def get(self, k): ...
    ....

def merge_sort(larik, kunci): ...

# ----- backend/m5_katalog.py -----
class Katalog:
    def tambah(self, nama, harga): ...
    def rentang(self, lo, hi): ...
    def stats(self): ...

class MinHeap:
    def push(self, kunci, nilai): ...
    def pop_min(self): ...
    def stats(self): ...

# ----- backend/m6_peta.py -----
class Peta:
    def zona(self, a, hops): ...
    def rute(self, a, b): ...
    ...
```

5 Data

Sample data di CSV seperti ini:

oid	pelanggan	resto	menu	harga	prioritas	t_masuk_detik	t_selesai_detik	status
O-000001	Rahmi Simanjuntak	kafe-teknik	roti-bakar	12000	3	47985		CANCEL
O-000002	Cici Handayani	kedai-kopi	bubur-ayam	11000	3	59780	62115	DONE
O-000003	Malika Dabukke	warung-bu-ida	mie-goreng	13000	3	39641	40026	DONE
O-000004	Oskar Puspita, S.Sos	kafe-teknik	nasi-padang	25000	2	29632		CANCEL
O-000005	R. Ratna Susanti, S.Pt	bakso-pak-yon	nasi-campur	20000	1	74921	77006	DONE
O-000006	Dr. Raina Firmansyah, S.I.Kom	kafe-teknik	ayam-geprek	19000	3	30415	31330	DONE
O-000007	Usyi Dongoran	warung-bu-ida	kentang-goreng	14000	2	66262	68612	DONE
O-000008	Tgk. Radika Tampubolon	kedai-kopi	iga-penyet	27000	2	31196	32651	DONE
O-000009	Hj. Ani Hartati, S.Pt	kantin-asrama	bubur-ayam	11000	3	67723	68563	DONE
O-000010	T. Rafid Usamah	kantin-pusat	sate-ayam	24000	3	40015	40765	DONE

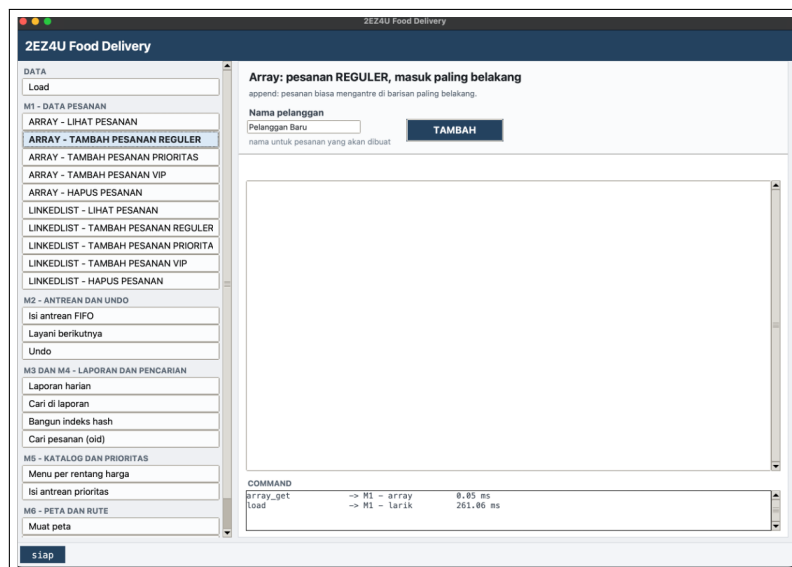
Kolom	Isi
oid	ID unik, 0-000001 sampai 0-200000
pelanggan, resto	data pelanggan dan restoran
menu, harga	data menu dan harga
prioritas	1 (VIP), 2 (Prioritas), 3 (Reguler)
t_masuk_detik	waktu pesanan masuk
t_selesai_detik	t_masuk + lama antar; kosong bila belum selesai
status	DONE / CANCEL / ANTRE

6 Antarmuka: Satu Jendela Tkinter

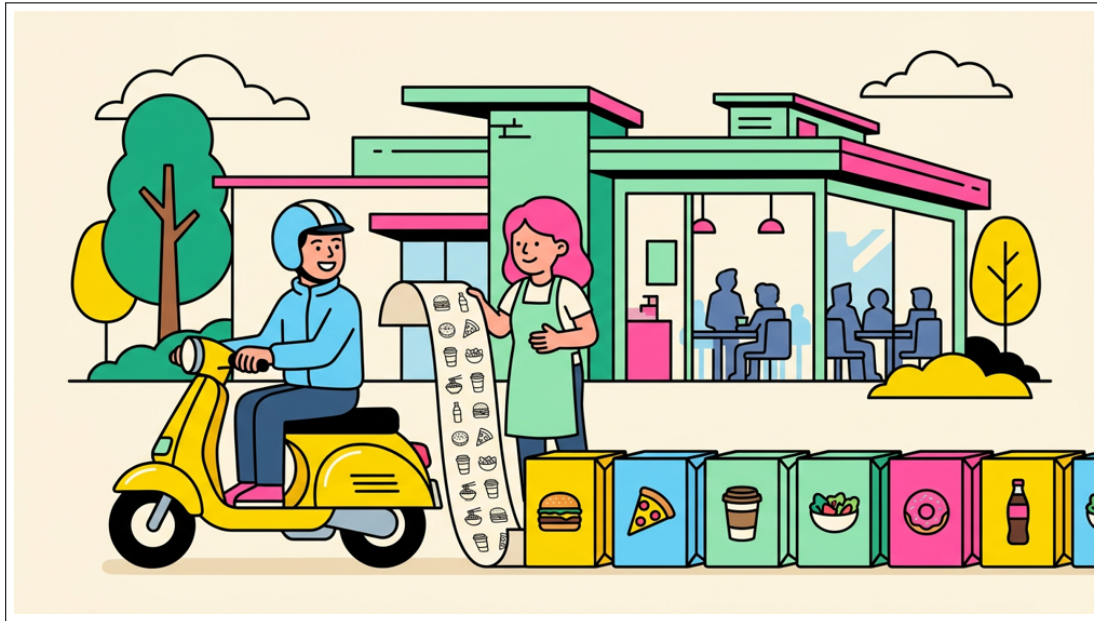
Antarmukanya menggunakan Tkinter

Contoh layout

Terdapat Tiga bagian: daftar menu di kiri, form parameter dan hasil di kanan, command di bawah. Gambar UI dibawah bisa menjadi referensi.



Contoh Menu ARRAY – TAMBAH PESANAN REGULER dipilih. Form di kanan hanya menampilkan parameter pada menu tersebut, panel COMMAND di bawah mencatat perintah yang sudah dijalankan beserta modul dan waktunya.



Ceritanya

Hari pertama layanan dibuka. Belum ada kurir, belum ada laporan, belum ada peta. Yang paling mendasar dulu: pesanan yang masuk harus **tersimpan dan tidak boleh hilang**, bisa **dibuka kembali** bila ada yang bertanya “pesanan nomor sekian isinya apa?”, bisa **ditambah**, dan bisa **dihapus**.

Yang membuat M1 tidak sederhana kelihatannya adalah aturan bisnisnya: pesanan datang dalam **tiga jenis**. Pesanan **REGULER** (3) mengantre di barisan paling belakang, pesanan **PRIORITAS** (2) menyerobot ke tengah, dan pesanan **VIP** (1) langsung ke urutan pertama. Tiga letak penyisipan yang berbeda inilah yang akan memperlihatkan kekuatan dan kelemahan dua struktur data paling dasar.

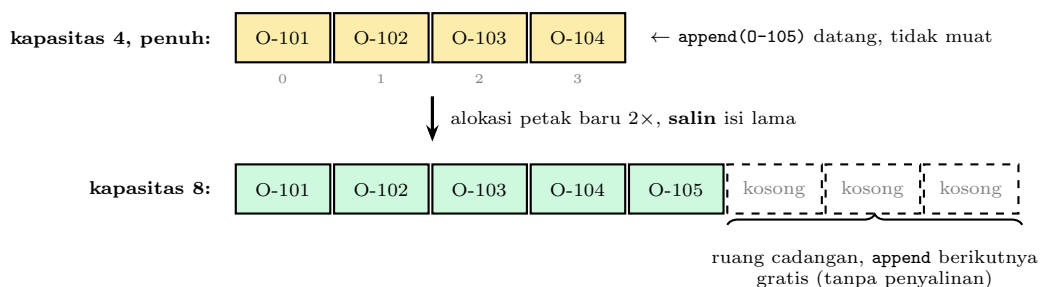
Backend M1

Data yang sama disimpan pada **dua struktur sekaligus**: sebuah array dan sebuah linked list. Keduanya harus melayani lima operasi yang persis sama, supaya performa bisa diadu langsung dari antarmuka.

Operasi	Artinya di aplikasi	Array	Linked List
Lihat pesanan ke- i	buka detail satu pesanan	$O(1)$	$O(n)$
Tambah REGULER	masuk barisan paling belakang	$O(1)$ amortized	$O(1)$
Tambah PRIORITAS	menyerobot ke tengah barisan	$O(n)$	$O(n)$ telusur
Tambah VIP	langsung ke urutan pertama	$O(n)$	$O(1)$
Hapus pesanan ke- i	pesanan dicabut dari barisan	$O(n)$	$O(n)$ telusur

Bagaimana Array Bekerja (1)

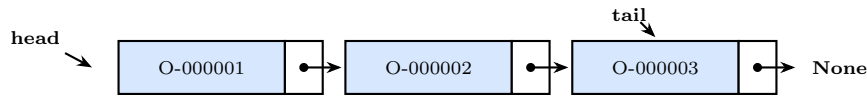
Sebuah array adalah sepetak memori berukuran **tetap**. Ketika penuh, ia tidak bisa “dilebarkan” begitu saja, yang dilakukan adalah menyiapkan petak baru berukuran dua kali lipat, menyalin seluruh isinya, lalu membuang petak lama.



Penyalinan itu mahal ($O(n)$), tetapi **jarang** terjadi: sekali setiap kapasitas berlipat. Bila biayanya dibagi rata ke seluruh **append**, hasilnya konstan. Inilah yang disebut **amortized** $O(1)$. Kalau kamu keliru menyalin *setiap* kali **append**, biayanya menjadi $O(n)$ per operasi dan $O(n^2)$ untuk seluruh pemuatan: benar secara hasil, tetapi **tumbang di uji skala**.

Bagaimana struktur Linked List Bekerja (2)

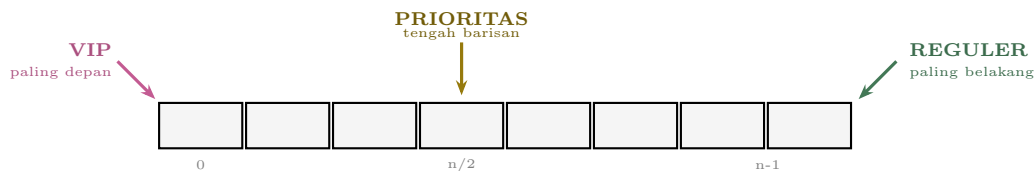
Linked List tidak memakai sepetak memori yang bersambung. Tiap pesanan dibungkus sebuah *simpul* yang menyimpan nilainya dan alamat simpul berikutnya. Karena itu tidak ada yang perlu digeser saat menyisipkan, tetapi juga tidak ada alamat yang bisa dihitung untuk menuju posisi ke- i .



penunjuk *tail* disimpan supaya menambah di belakang tetap $O(1)$

Tiga jenis pesanan, tiga letak penyisipan

Di sinilah dua struktur tadi diadu. Aplikasi menerima tiga jenis pesanan, dan yang membedakan biayanya bukan isi pesannya, melainkan **di mana ia disisipkan**.



Angka berikut diambil dari implementasi referensi pada data 200.000 baris. Perhatikan kolom PRIORITAS: keduanya hampir seimbang, tetapi **karena sebab yang berbeda**.

Jenis	Array	Linked List	Apa yang sebenarnya terjadi
REGULER	0,02 ms	0,01 ms	keduanya cukup menempel di ujung belakang
PRIORITAS	3,71 ms	2,19 ms	array menggeser 100.000 elemen; linked list menelusuri 100.000 simpul lalu menyambung sekali
VIP	6,82 ms	0,00 ms	array menggeser seluruh isi; linked list hanya mengubah satu penunjuk

Fitur M1

Fitur Array Method	Linked List Method	Keterangan
ARRAY - LIHAT PESANAN	LINKED LIST - LIHAT PESANAN	Melihat pesanan yang tersedia
ARRAY - TAMBAH PESANAN REGULER	LINKED LIST - TAMBAH PESANAN REGULER	Menambahkan pesanan biasa
ARRAY - TAMBAH PESANAN PRIORITAS	LINKED LIST - TAMBAH PESANAN PRIORITAS	Menambahkan pesanan prioritas
ARRAY - TAMBAH PESANAN VIP	LINKED LIST - TAMBAH PESANAN VIP	Menambahkan pesanan VIP
ARRAY - HAPUS PESANAN	LINKED LIST - HAPUS PESANAN	Menghapus pesanan dari antrean